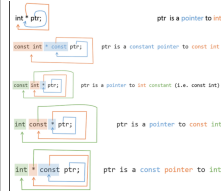


Cheat Sheet

Wednesday, March 12, 2025 11:33 PM

Week 2	Const	C array	Memory Models
String operations	- immutable	char s[6];	- stack
insert()	- means we cannot change	s[0] = 'h';	o local variables
erase()	Functions	s[1] = 'e';	o allocation = automatic
resize()	- vector to a function		o deallocation = when function exists
substr()	- pass by value vector<int> vec		- static
contains()	o copied		o alloc= at compile
ends_with()	- pass by reference vector<int> & vec		o dealloc = when program terminates
starts_with()	o no copy		- heap
	- pass by const reference const vector<int> & vec		o allocate = int *p = new int (42);
Numeric Types	o no copy and cannot be changed in the func		o deallocate = delete p;
- char [1], short [2],		C++ pointers	
int [4], long [4, 8], long		- gets address of something	
long [8] float [4]		- dereference address to get to what it's pointing to	
double [8]		o *p = &x	
- [signed] and		o int a=0; int *p = &a; (*p)++; int x = a;	
[unsigned]		o dereference above makes use increment a	
- bool [1]			

Week 3	Casting	Exceptions (try, catch)
spiral rule	- static_cast	- exception must be value
	o conversions safe for compiler	o int, float, char
	o between related types	o string
	o static_cast<new_type>(expression)	o any object
	o ex int to double	o pointer to something
	- dynamic_cast	- report exceptions
	o downcasting	o throw <expr> errr
	o proly don't use	▪ a statement
	- const_cast	▪ value of
	o removing const	expression
	o only use if underlying data mutable	thrown
	o const int a = 10 ;	- handle exceptions
	o int* modifiable = const_cast<int*>(&a);	o catch
	- reinterpret_cast	
	o convert the pointer to any other type of pointer	
	o reinterpret_cast<new_type> (expression)	
References		
- only exist c++		
- serves same purpose as		
pointers		
- helps with copying saving		
data space		
	Control Flow c++	
	- try-catch-throw	
	- range loop	
	- intervals	

Week 1
C++ class based
A class is treated like a Type
Object is instance of a class

Week 4
STL
- containers
o sequence
▪ vector
□ sequence
□ dynamic size
□ add/remove anywhere
□ bounds safety
▪ list
▪ deque
▪ arrays
▪ forward_list - linked list
o adaptors
▪ queue, priority queue,
▪ stack
□ LIFO
□ encapsulated object of vector or dequeue
o associative - O(logn)
▪ set - balance binary tree
▪ multiset - bst
▪ map - bst
▪ multimap - bst
o unordered associative
▪ unordered set, multiset, map, multimap - hashmap
- iterators
o point at begin and end move forward backward
o right place - front end
o direction - forward backward
o iterator
o reverse iterator
o can't insert at .end()
o back_inserter() could work
- algorithms
o auto- automatically finds type

Week 6	Polymorphism - object behave different based of type/class
Inheritance	- compile-time
- promote code reuse	o function overloading
- theory	o operator overloading
o support type refinements	- runtime poly
o subtyping through subclassing	o ability for object to respond based of dynamic type rather than compile time type
- A <: B = A is subtype of B	o function overriding and virtual function
o B is expected you can provide A	o virtual switches from static to dynamic
Inheritance and privacy	- ex: runtime
- can alter privacy of what you want to inherit	class Mammal {
o public	public:
▪ public - public	Mammal() {}
▪ protected - protected	virtual void print() {
▪ private - private	std::cout << "I (" << this << ")'m a mammal!" << std::endl;
▪ when overrided use above setting	}
▪ when A inherits B everyone knows	};
o protects	class Human : public Mammal {
▪ public- protected	public:
▪ protected - private	Human() {}
▪ when override use above privacy setting	virtual void print() { //if no virtual here still treated like it has it
▪ when A inherits B only A and subclasses know	std::cout << "I (" << this << ")'m a human!" << std::endl;
o private	}
▪ public - private	};
▪ protected - private	Final
▪ privacy setting using when override called	- cannot be override after override final, this would be the last override
□ only A and A only knows it inherits from B	- class B { public: virtual void f(int) {std::cout << "B::f" << std::endl;};
o class Human: public Mammal - inheritance	virtual void f(int) override final {std::cout << "D::f" << std::endl;}}
o when inheritance upgrades/refines methods it inherits that means overriding	Deallocation
o dynamic binding - polymorphism	- make destructor polymorphic
o	- virtual ~Base
	- must also delete poly borrower ~Derived
	Multiple Inheritance Diamond
	- parent classes will inherit virtual
	- calls Student: virtual public person
	- this would avoid confusion with the compile where the class pulls shared methods